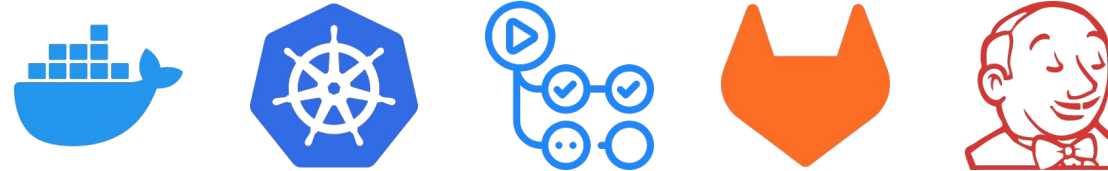


DEVOPS FUNDAMENTALS SERIES

DevOps Fundamentals: Docker, Kubernetes, and CI/CD

A practical, beginner-friendly introduction to the modern delivery pipeline



Presented by: Eng Abdallah Elmalawany · Eng Mahmoud Sharif



What We'll Cover Today

01

Introduction to DevOps

culture, lifecycle, and core practices

02

Containers and Docker

images, Dockerfiles, and security

03

Kubernetes for Beginners

architecture and core objects

04

CI/CD Automation

pipelines, GitHub Actions, GitLab, Jenkins

05

Bringing It Together

one real deployment, start to finish



SECTION 1 / 5

Introduction to DevOps

Why organizations changed how they build and ship software.



What is DevOps?

01

Collaboration

breaks down the wall between
Development and Operations teams

02

Automation

replaces manual, error-prone steps with
repeatable pipelines

03

Fast feedback

problems are found and fixed in minutes,
not weeks

04

Shared ownership

developers care about production; ops cares about the code

05

Reliability

the goal is to ship often, safely, and predictably



Traditional Software Delivery Problems

01

Dev vs Ops silos

two teams with different goals and little communication

02

Manual deployments

slow, repetitive, and prone to human error

03

Environment inconsistency

"it works on my machine" but fails on the server

04

Slow releases

weeks or months between versions reaching users

05

Difficult rollback

no safe, fast way to undo a bad release



DevOps Culture and Lifecycle



01

Plan



02

Code



03

Build



04

Test



05

Release



06

Deploy



07

Operate



08

Monitor

LOOP

Continuous Feedback Loop



Core DevOps Practices

01

Version control

Git as the single source of truth for code and config

02

Infrastructure as Code

define servers and clusters in versioned files

03

Continuous Integration

merge and test code changes frequently

04

Continuous Delivery/Deployment

automatically ship tested changes

05

Automated testing

catch regressions before they ever reach a user



A Real-World DevOps Workflow



01

Developer pushes code



02

CI pipeline runs tests



03

Docker image built & scanned



04

Image pushed to registry



05

Kubernetes deploys new version



06

Monitoring validates release



SECTION 2 / 5

Containers and Docker

Package once, run anywhere — solving the environment problem.



What is Docker?

Docker is an open-source platform that packages an application together with everything it needs to run: the code, the runtime and all its libraries, in one image.

That image runs as a container, an isolated process that shares the kernel of the host instead of starting an operating system of its own. A container starts in seconds, stays small, and behaves the same way on a laptop, on a build server and in production.

You use two parts. The Docker CLI is the command you type, and the Docker Engine builds the image, stores it and runs the container for you.

`build once` · `ship the image` · `run it anywhere`



Installing Docker



Windows / macOS

- 01 On Windows you need Windows 10 or 11 with WSL 2 enabled
- 02 Download Docker Desktop and run the installer
- 03 Docker Desktop includes the Engine, the CLI and Compose
- 04 Check the install in a terminal: `docker --version`



Linux (Ubuntu)

- 01 Quick install: `curl -fsSL https://get.docker.com | sh`
- 02 Or add the Docker apt repository, then: `apt install docker-ce`
- 03 To work without sudo: `usermod -aG docker $USER`, then sign out and in
- 04 Check the install: `docker run hello-world`



Virtualization vs Containerization



ASPECT

VIRTUAL MACHINE

CONTAINER

What is isolated

A full operating system per app

A single process on the host

Start-up time

Minutes, an OS has to boot

Seconds, a process starts

Typical size

Gigabytes

Megabytes

How many per server

A few virtual machines

Dozens of containers

Isolation strength

Very strong, hardware level

Good, but the kernel is shared

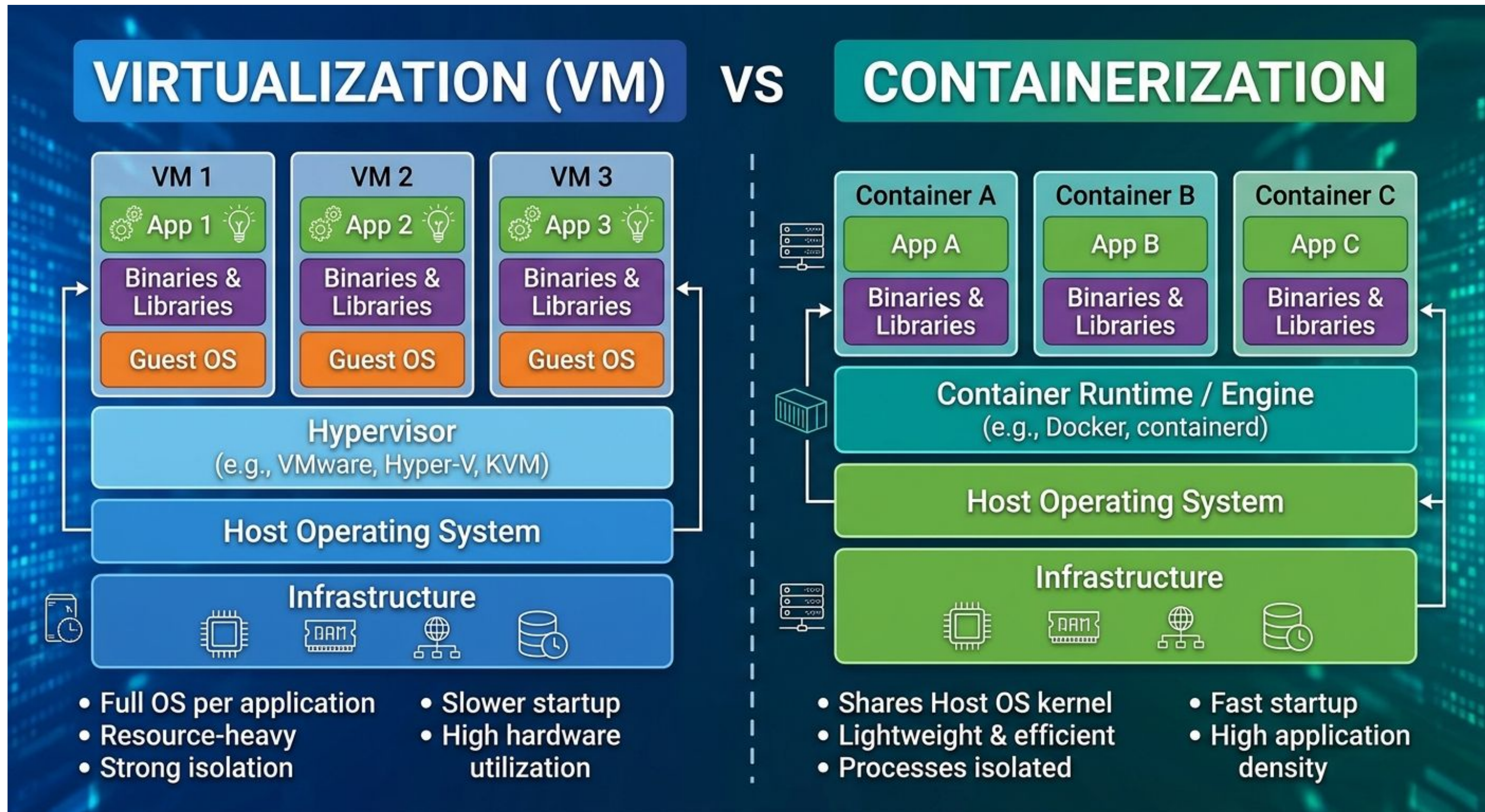
Best used for

Running a different OS

Microservices, CI/CD, scaling

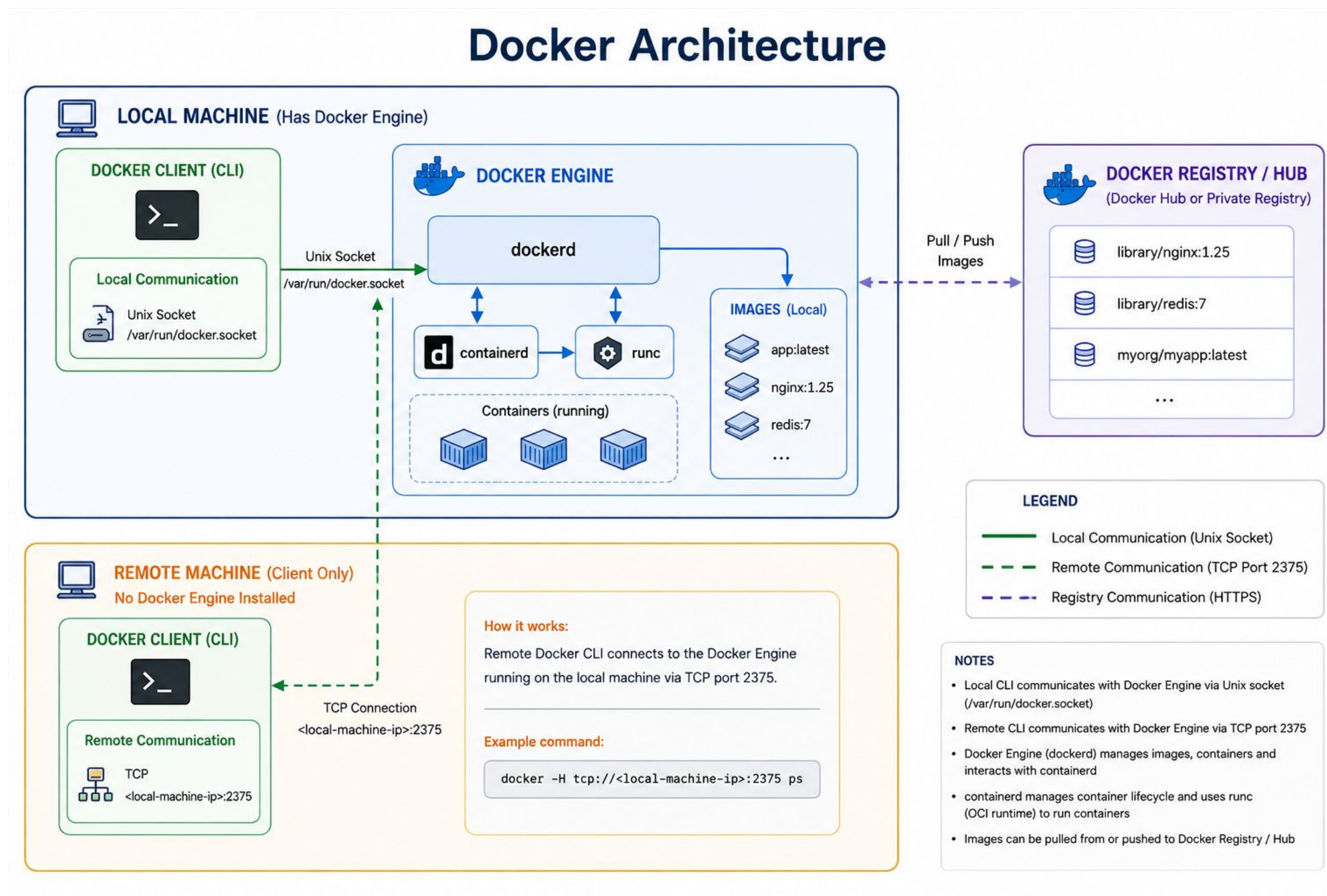


Two Ways to Isolate an App





Docker Architecture



How a command flows

- 01 The CLI sends your command to the Docker daemon
- 02 The daemon builds images and manages containers
- 03 `containerd` and `runc` start the container process
- 04 A registry stores images, ready to pull or push



Inside the Docker Engine

01

Docker CLI

the docker command you type; it sends every request to the daemon

02

dockerd

the daemon that builds images and manages containers, networks and volumes

03

containerd

keeps track of each container from start to stop

04

runc

creates the container process, with its namespaces and limits

05

Images and registry

image layers are stored on disk, and a registry shares them between machines



Namespaces: What a Container Sees

01

PID

only its own processes, and your app is process number 1

02

NET

its own network interface, IP address and ports

03

MNT

its own file system, built from the image layers

04

UTS and IPC

its own host name, and its own shared memory

05

USER

root inside the container can be a normal user on the host



Cgroups: What a Container Can Use

01

CPU

--cpus=1.5 limits how much processor time it may use

02

Memory

--memory=512m; the container is stopped if it goes over

03

Block I/O

limits how much it may read from and write to the disk

04

Processes

--pids-limit=200 keeps one container from flooding the host

05

Namespaces and cgroups

namespaces decide what a container can see, cgroups decide how much it can use



Image and Container: Layers

IMAGE

read-only layers, built once

LAYER 4

```
COPY . . + CMD ["python", "app.py"]
```

LAYER 3

```
RUN pip install -r requirements.txt
```

LAYER 2

```
COPY requirements.txt .
```

LAYER 1

```
FROM python:3.12-slim
```

`docker build` → layers that never change

Many containers can share one image. Data you want to keep belongs in a volume, not in the writable layer.

CONTAINER

the same layers plus one writable layer

WRITABLE LAYER

logs, temporary files, anything the app writes

LAYER 4

```
COPY . . + CMD ["python", "app.py"]
```

LAYER 3

```
RUN pip install -r requirements.txt
```

LAYER 2

```
COPY requirements.txt .
```

LAYER 1

```
FROM python:3.12-slim
```

`docker run` → a thin layer, removed with the container



Image and Container Commands



COMMAND

WHAT IT DOES

```
docker pull nginx:1.27
```

download an image from a registry

```
docker images
```

list the images stored on this machine

```
docker build -t myapp:1.0 .
```

build an image from the Dockerfile in this folder

```
docker run -d -p 8080:80 myapp:1.0
```

start a container in the background and publish a port

```
docker ps -a
```

list all containers, running and stopped

```
docker logs -f <id>
```

watch the output of a container as it runs

```
docker exec -it <id> sh
```

open a shell inside a running container

```
docker rm -f <id> · docker rmi <img>
```

delete a container · delete an image



Docker Hub and Image Tags

01

Registry

where images are stored; Docker Hub is the public one, and private ones work the same way

02

Repository

a named group of images, for example myorg/myapp

03

Tag

the version label on an image, for example myapp:1.4.2

04

latest is not the newest

it is only the default tag and it keeps moving, so always name a real version in production

05

Digest

myapp@sha256:... points to one exact image, so every deployment is identical



Working with a Registry

COMMAND

```
docker login
```

```
docker tag myapp:1.0 me/myapp:1.0
```

```
docker push me/myapp:1.0
```

```
docker pull me/myapp:1.0
```

```
docker image inspect me/myapp:1.0
```

```
docker search postgres
```

WHAT IT DOES

sign in to Docker Hub or a private registry

give the image a name that points to your repository

upload the image and its layers

download the image on another machine

show the digest, the layers and the settings

search public images from the command line



Docker Network Types

01

bridge

the default private network on one host;
publish ports with -p 8080:80

02

user-defined bridge

the same, and containers can also reach
each other by name

03

host

no network isolation; the container uses
the network of the host

04

none

no network at all, for tasks that work offline

05

overlay and macvlan

overlay joins containers across hosts; macvlan gives a container
its own LAN address



Network Commands



COMMAND

WHAT IT DOES

```
docker network ls
```

list the networks on this machine

```
docker network create appnet
```

create your own bridge network with DNS

```
docker run --network appnet ...
```

connect a container when it starts

```
docker network connect appnet web
```

connect a container that is already running

```
docker network inspect appnet
```

show the subnet and the connected containers

```
docker run -p 8080:80 nginx
```

publish a container port on the host



Docker Storage Options

01

Volume

storage managed by Docker that stays after the container is removed. It is the normal choice for databases and any data you want to keep.

02

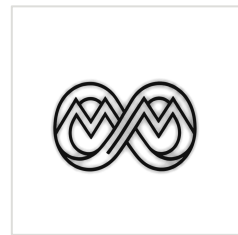
Bind mount

a folder from the host, mounted into the container. Useful while developing, because your edits appear inside the container at once.

03

tmpfs

storage that lives in memory only and never reaches the disk. A good place for short-lived files and secrets.



Storage Commands



COMMAND

WHAT IT DOES

```
docker volume create appdata
```

create a named volume

```
docker volume ls · inspect appdata
```

list volumes · show where the data is kept

```
docker run -v appdata:/var/lib/data
```

mount that volume inside a container

```
docker run -v $(pwd):/app
```

mount the current folder for development

```
docker run --tmpfs /tmp:size=64m
```

mount a folder that lives in memory

```
docker volume prune
```

delete the volumes no container uses



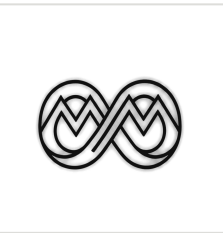
Dockerfile: A Python App

Dockerfile x

```
01  # a small base image with a fixed version
02  FROM python:3.12-slim
03
04  WORKDIR /app
05
06  # install the dependencies first, so this layer is reused
07  COPY requirements.txt .
08  RUN pip install --no-cache-dir -r requirements.txt
09
10  # then copy the application code
11  COPY . .
12
13  ENV PORT=8000
14  EXPOSE 8000
15  USER 1000
16  CMD ["python", "app.py"]
```

main

Ln 1, Col 1 Spaces: 2 UTF-8



Dockerfile Keywords



KEYWORD

WHAT IT DOES

FROM

the base image that every later step builds on

WORKDIR

the folder the next instructions run in

COPY · ADD

copy files into the image; ADD can also unpack archives

RUN

runs a command during the build and saves the result as a layer

ENV · ARG

a variable at run time · a variable during the build

EXPOSE

states which port the application listens on

USER

the user the application runs as; choose a user other than root

CMD · ENTRYPOINT

the default command · the fixed program to start



Faster Builds: Order Your Layers

slow.Dockerfile ×

```
01 FROM python:3.12
02
03 # the whole project is copied first
04 COPY . .
05 RUN pip install -r requirements.txt
06
07 CMD ["python", "app.py"]
08
09 # any change to any file breaks this
10 # layer, so every build installs all
11 # the dependencies again
```

main

Ln 1, Col 1 Spaces: 2 UTF-8

fast.Dockerfile ×

```
01 FROM python:3.12-slim
02
03 # the dependency list comes first
04 COPY requirements.txt .
05 RUN pip install --no-cache-dir \
06     -r requirements.txt
07
08 # the code comes last, it changes most
09 COPY . .
10 CMD ["python", "app.py"]
11
12 # the install layer is reused until
13 # requirements.txt changes
```

main

Ln 1, Col 1 Spaces: 2 UTF-8



Smaller, Faster Builds

01

.dockerignore

keep .git, virtual environments, tests and data files out of the build

02

Order the layers

copy the dependency files before the source code, so the install stays cached

03

Small base images

python:3.12-slim or alpine: less to download, and less to scan

04

Multi-stage builds

build in one stage and copy only the finished program into a clean image

05

BuildKit cache mounts

RUN --mount=type=cache keeps the package cache between builds



Docker Security Essentials

01

Trusted base images

use official images with a fixed version, and never build on :latest

02

Scan every image

run Trivy in the pipeline and stop the build on high or critical findings

03

Never run as root

set a USER in the Dockerfile, and add --read-only and --cap-drop=ALL when running

04

Keep secrets out of images

read them at run time from Vault or Compose secrets; a secret inside an image is public

05

Give the least privilege

avoid --privileged, do not mount the Docker socket, and always set resource limits



Security in Practice

scan.sh x

```
01  # stop on the findings that matter
02  trivy image --exit-code 1 \
03    --severity HIGH,CRITICAL myapp:1.0
04
05  # check the Dockerfile itself
06  trivy config .
07
08  # list the packages in the image
09  docker sbom myapp:1.0
10
11  # check which user the app runs as
12  docker run --rm myapp:1.0 id
```

main

Ln 1, Col 1 Spaces: 2 UTF-8

run-hardened.sh x

```
01  # read the secret at run time
02  DB_PASS=$(vault kv get -field=pass app/db)
03
04  docker run -d \
05    --user 1000:1000 \
06    --read-only \
07    --cap-drop=ALL \
08    --security-opt no-new-privileges \
09    --memory=512m --pids-limit=200 \
10    -e DB_PASS \
11    myapp:1.0
```

main

Ln 1, Col 1 Spaces: 2 UTF-8



Docker Compose: One File, Whole Stack

docker-compose.yml ×

```
01  services:
02    web:
03      build: .
04      ports: ["8000:8000"]
05      environment:
06        DB_HOST: db
07      depends_on: [db]
08
09    db:
10      image: postgres:16
11      environment:
12        POSTGRES_PASSWORD: secret
13      volumes:
14        - dbdata:/var/lib/postgresql/data
15
16  volumes:
17    dbdata:
```

main

Ln 1, Col 1 Spaces: 2 UTF-8



Compose Commands



COMMAND

WHAT IT DOES

```
docker compose up -d
```

start the whole stack in the background

```
docker compose ps
```

list the services and their state

```
docker compose logs -f web
```

follow the output of one service

```
docker compose exec web sh
```

open a shell in a running service

```
docker compose build --no-cache
```

rebuild the images from scratch

```
docker compose down
```

stop and remove the containers and networks

```
docker compose down -v
```

the same, and also delete the named volumes



SECTION 3 / 5

Kubernetes for Beginners

Running containers reliably, at scale, across many machines.



Why Docker Alone Is Not Enough in Production

01

Multiple containers, multiple servers

Docker alone doesn't coordinate across machines

02

Self-healing

who restarts a container when it crashes at 3am?

03

Scaling

how do you run more copies when traffic spikes?

04

Load balancing & service discovery

how do containers find and share traffic with each other?

05

Rolling updates & config/secret management

how do you deploy safely and manage configuration at scale?



What is Kubernetes?

01

Container orchestration platform

open-source, originally built at Google

02

Automates

deployment, scaling, and management of containerized workloads

03

Declarative desired state

you describe what you want; Kubernetes makes it happen

04

Self-healing by design

it constantly works to match reality to your declaration

05

Often abbreviated

as "k8s" (8 letters between the K and the s)



Kubernetes Cluster Architecture



Control Plane

API Server

front door for all cluster requests

Scheduler

decides which node runs each Pod

Controller Manager

keeps actual state = desired state

etcd

cluster's key-value data store



Worker Node(s)

kubelet

talks to the API server, runs Pods

kube-proxy

handles networking rules per node

Container runtime

actually runs the containers

Pods

your running application workloads



Bootstrapping a Cluster with kubeadm

control plane node ×

```
01 # 1. Install container runtime, kubeadm, kubelet, kube
02 #   on every node first (control plane + workers)
03
04 # 2. Initialize the control plane
05 kubeadm init \
06   --pod-network-cidr=10.244.0.0/16
07
08 # 3. Configure kubectl for your user
09 mkdir -p $HOME/.kube
10 cp /etc/kubernetes/admin.conf $HOME/.kube/config
11
12 # 4. Install a pod network add-on (CNI)
13 kubectl apply -f <cni-provider.yaml>
```

main

Ln 1, Col 1 Spaces: 2 UTF-8

worker nodes ×

```
01 # 5. On each worker node, run the join
02 #   command printed by 'kubeadm init'
03 kubeadm join <control-plane-host>:6443 \
04   --token <token> \
05   --discovery-token-ca-cert-hash sha256:<hash>
06
07 # 6. Verify from the control plane
08 kubectl get nodes
09 kubectl get pods -A
```

main

Ln 1, Col 1 Spaces: 2 UTF-8



Minikube: A Local Kubernetes Cluster

minikube – easiest way to start ×

```
01 # 1. Install minikube (see docs for your OS)
02
03 # 2. Start a local single-node cluster
04 minikube start
05
06 # 3. Use kubectl exactly like a real cluster
07 kubectl get nodes
08 kubectl apply -f deployment.yaml
09
10 # 4. Open the built-in dashboard
11 minikube dashboard
12
13 # 5. Stop or delete when done
14 minikube stop
15 minikube delete
```

main

Ln 1, Col 1 Spaces: 2 UTF-8



Kubernetes Core Objects

01

Pod

the smallest deployable unit, wrapping one or more containers

02

Deployment & ReplicaSet

manage replicas, rollouts, and self-healing

03

Service & Ingress

provide stable networking and external access

04

Namespace

a virtual sub-cluster for organizing resources

05

ConfigMap, Secret, PV/PVC

configuration, sensitive data, and persistent storage



What is a Pod?

01

Smallest deployable unit

in Kubernetes, you never deploy a container directly

02

Usually one main container

though a Pod can hold small helper (sidecar) containers

03

Shared context

containers in the same Pod share an IP, storage, and network namespace

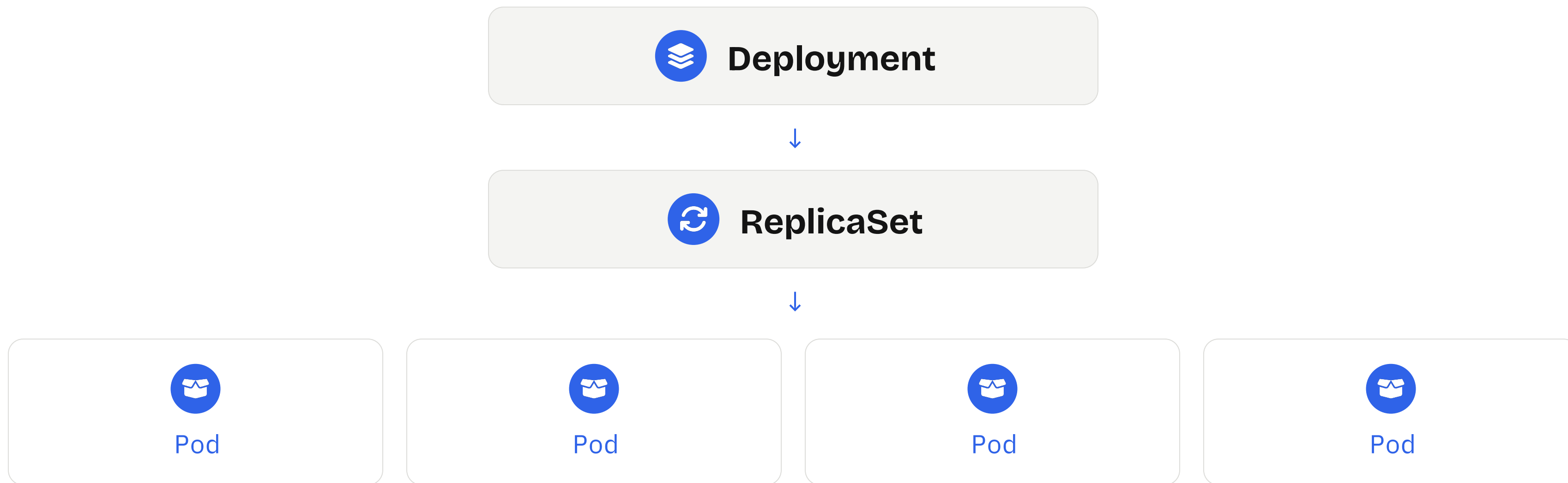
04

Lifecycle

Pods are created, run, and are eventually replaced — never repaired in place



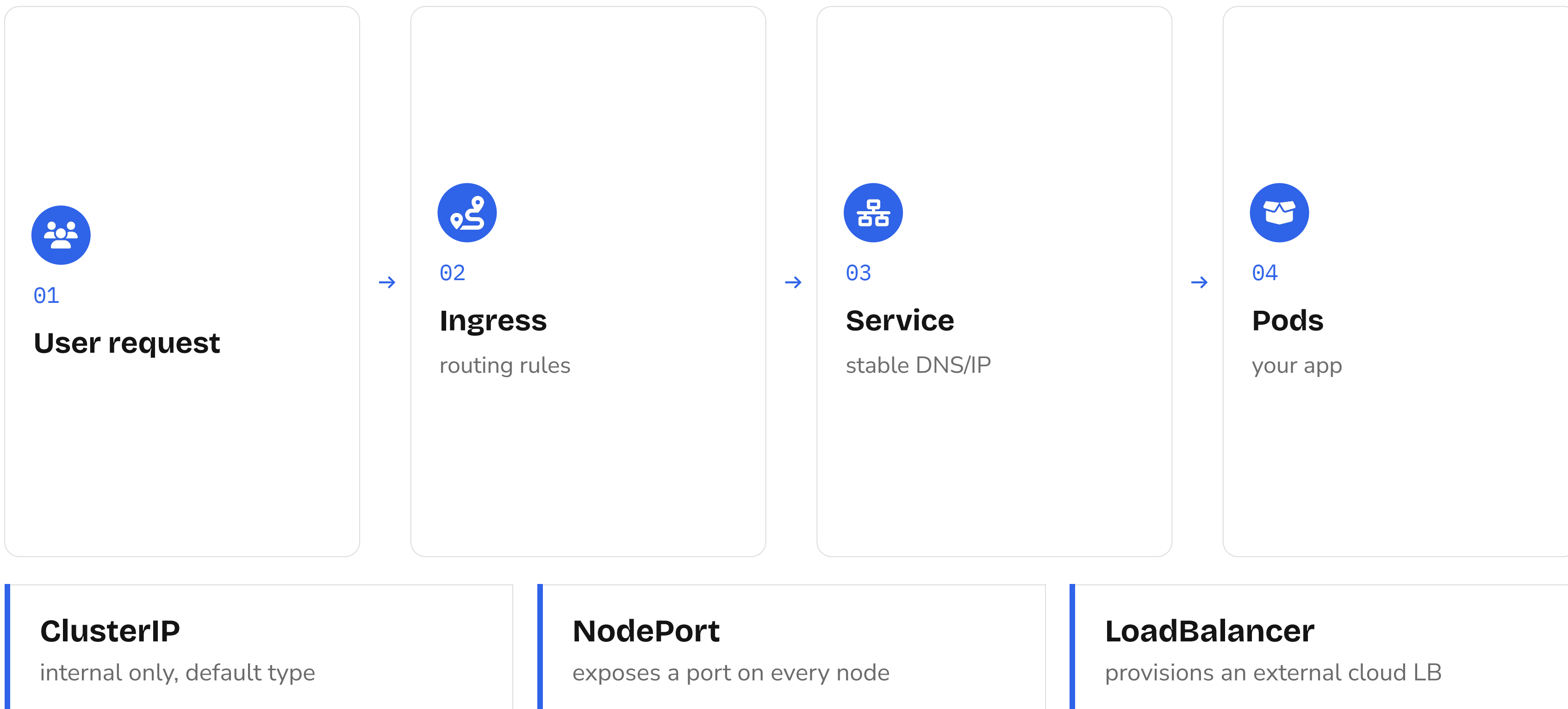
Deployment and ReplicaSet



Deployment defines the desired state → ReplicaSet keeps the right number of Pods running → Pods run your application



Kubernetes Services and Networking





ConfigMaps, Secrets, and Storage

01

Never hardcode config

into a container image; inject it instead

02

ConfigMap

stores non-sensitive configuration (URLs, feature flags)

03

Secret

stores sensitive values (passwords, tokens, keys)

04

PersistentVolume / Claim

give Pods durable storage that outlives them

05

Important

Secrets are base64-encoded, not encrypted — that's obfuscation, not security



Kubernetes Manifest Example: Deployment

deployment.yaml ×

```
01  apiVersion: apps/v1
02  kind: Deployment
03  metadata:
04    name: myapp
05  spec:
06    replicas: 3
07    selector:
08      matchLabels: { app: myapp }
09  template:
10    metadata:
```

main

Ln 1, Col 1 Spaces: 2 UTF-8



Kubernetes Manifest Example: Deployment (continued)

deployment.yaml x

```
11     labels: { app: myapp }
12 spec:
13   containers:
14     - name: myapp
15       image: myrepo/myapp:1.0
16       ports:
17         - containerPort: 3000
18       resources:
19         requests: { cpu: "100m", memory: "128Mi" }
20         limits:   { cpu: "250m", memory: "256Mi" }
```

main

Ln 1, Col 1 Spaces: 2 UTF-8



Exposing Apps with Ingress

ingress.yaml x

```
01  apiVersion: networking.k8s.io/v1
02  kind: Ingress
03  metadata:
04    name: myapp-ingress
05  spec:
06    rules:
07      - host: myapp.example.com
08        http:
09          paths:
10            - path: /
11              pathType: Prefix
12              backend:
13                service:
14                  name: myapp-service
15                  port: { number: 80 }
```

main

Ln 1, Col 1 Spaces: 2 UTF-8



Scaling and Self-Healing in Action

01

Manual scaling

```
kubectl scale deployment myapp --replicas=5
```

02

Horizontal Pod Autoscaler

automatically scales replicas based on CPU/memory or custom metrics

03

Self-healing

a crashed Pod is automatically replaced within seconds

04

Rolling updates

new Pods start and old ones terminate gradually, with zero downtime

05

Rollback

```
kubectl rollout undo
```

 instantly reverts to the previous working version



SECTION 4 / 5

CI/CD Automation

Turning every code change into a safe, automatic release.



What is CI/CD?

01

CI — Continuous Integration

developers merge code frequently, and it's automatically built and tested

02

CD — Continuous Delivery

every change that passes tests is automatically packaged, ready to release

03

CD — Continuous Deployment

a further step: passing changes are released to production automatically

04

Pipeline

the automated sequence of stages a change moves through

05

Goal

small, frequent, low-risk releases instead of big, risky ones



CI vs Delivery vs Deployment

01

Continuous Integration

automated build + test on every commit; stops broken code early

02

Continuous Delivery

adds automated packaging and staging; a human clicks "deploy"

03

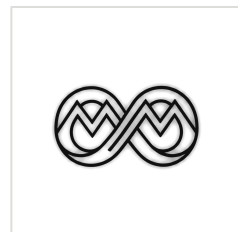
Continuous Deployment

removes the manual click — every passing change goes live automatically

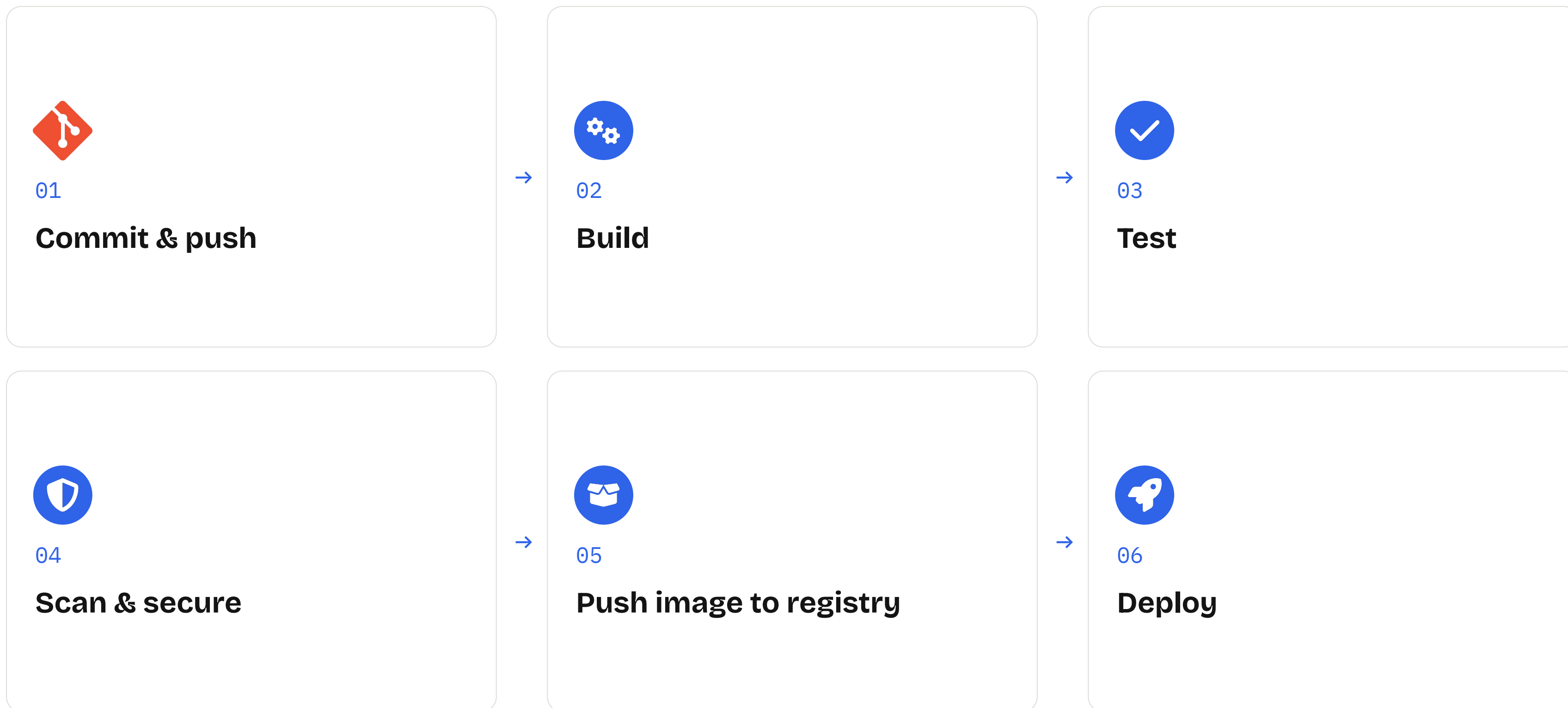
04

Choosing between them

regulated or high-risk systems often keep a manual approval gate



A Typical CI/CD Pipeline





GitHub Actions Overview

■ .github/workflows/ci.yml ×

```
01 name: CI
02 on: [push]
03 jobs:
04   build:
05     runs-on: ubuntu-latest
06     steps:
07       - uses: actions/checkout@v4
08       - uses: docker/build-push-action@v6
09         with:
10           push: true
11           tags: myrepo/myapp :${{ github.sha }}
```

main

Ln 1, Col 1 Spaces: 2 UTF-8



GitLab CI/CD Overview

■ .gitlab-ci.yml ×

```
01 stages: [build, test, deploy]
02
03 build:
04   stage: build
05   script:
06     - docker build -t myapp:$CI_COMMIT_SHA .
07
08 test:
09   stage: test
10   script: [npm test]
11
12 deploy:
13   stage: deploy
14   script: [kubectl apply -f k8s/]
15   when: manual
```

main

Ln 1, Col 1 Spaces: 2 UTF-8



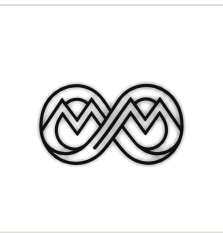
Jenkins Overview

Jenkinsfile x

```
01 pipeline {
02   agent any
03   stages {
04     stage('Build') {
05       steps { sh 'docker build -t myapp .' }
06     }
07     stage('Test') {
08       steps { sh 'npm test' }
09     }
10     stage('Deploy') {
11       steps { sh 'kubectl apply -f k8s/' }
12     }
13   }
14 }
```

main

Ln 1, Col 1 Spaces: 2 UTF-8



GitHub Actions vs GitLab CI/CD vs Jenkins

	 GITHUB ACTIONS	 GITLAB CI/CD	 JENKINS
Hosting	Managed by GitHub	Managed (or self-hosted)	Self-hosted only
Setup effort	Very low	Low	Higher — you run the server
Config format	YAML workflows	Single YAML file	Groovy Jenkinsfile
Best fit	Teams already on GitHub	All-in-one GitLab platform	Complex, custom pipelines
Extensibility	Marketplace of Actions	Built-in + integrations	Huge plugin ecosystem



SECTION 5 / 5

Bringing It All Together

One real deployment workflow, start to finish.



BRINGING IT ALL TOGETHER



Docker + Kubernetes + CI/CD: End to End



01

Dev pushes to Git



02

CI runs tests



03

Docker builds & scans image



04

Image pushed to registry



05

CD updates K8s manifests



06

Cluster runs & monitors app



BRINGING IT ALL TOGETHER



Key Takeaways

01

DevOps

is culture and automation working together, not a single tool

02

Docker

packages an app once so it runs identically everywhere

03

Kubernetes

keeps containers running reliably, at scale, across many machines

04

CI/CD

turns every commit into a small, safe, automated release

05

Together

they form one continuous loop: code → build → ship → run → learn



BRINGING IT ALL TOGETHER



Recommended Next Steps

01

Practice Docker

containerize one of your own small projects this week

02

Try Kubernetes locally

with Minikube or kind, no cloud account required

03

Build a pipeline

set up a free GitHub Actions workflow on a personal repo

04

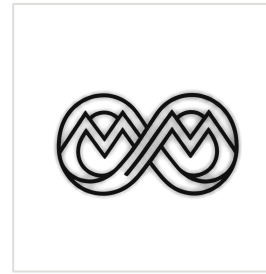
Read the official docs

Docker Docs and the Kubernetes documentation are excellent

05

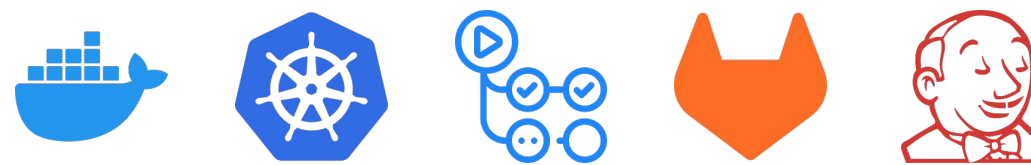
Keep it small

one working end-to-end pipeline beats memorizing every feature



QUESTIONS & DISCUSSION

Thank You



Eng Abdallah Elmalawany · Eng Mahmoud Sharif